

Práctica 3: Implementación de Tareas Periódicas y Concurrentes de Tiempo Real

1. Preguntas orientadoras

1.1. Diferencias entre temporizadores y relojes POSIX-RT

Un reloj (*clock*) proporciona una referencia temporal del sistema. Puede consultarse con `clock_gettime()` o utilizarse para esperar hasta un instante específico mediante `clock_nanosleep()`. En esta práctica se usa `CLOCK_MONOTONIC`, cuyo avance no depende de ajustes del reloj civil.

Un temporizador (*timer*) se crea sobre un reloj para notificar cuando vence un plazo, una sola vez o de forma periódica. Se configura con `timer_create()` y `timer_settime()`; la expiración puede comunicarse, por ejemplo, mediante una señal. Por tanto, el reloj es la base de tiempo y el temporizador programa notificaciones sobre esa base.

En el sistema implementado, Tau 1 y Tau 2 usan temporizadores POSIX que notifican mediante señales en tiempo real. Tau 3 y Tau 4 utilizan el reloj monotónico y esperan sus fechas de activación con `clock_nanosleep()` en modo absoluto (`TIMER_ABSTIME`).

1.2. Rol de las llamadas al sistema, los eventos y las señales

Las llamadas al sistema y las interfaces POSIX permiten consultar el tiempo, programar esperas, crear hilos y sincronizar su ejecución. Un evento —por ejemplo, la expiración de un temporizador— hace que un hilo que estaba esperando pueda continuar. Las señales son un mecanismo para notificar algunos de esos eventos; no reemplazan el cuerpo de la tarea. Cuando un hilo queda listo, el planificador del sistema operativo determina cuándo obtiene el procesador.

En esta implementación, las señales `SIGRTMIN+1` y `SIGRTMIN+2` notifican las expiraciones de los temporizadores de Tau 1 y Tau 2. Los hilos esperan dichas señales con `sigwait()`. Tau 3 y Tau 4 esperan fechas absolutas mediante `clock_nanosleep()`. Mutexes y variables condicionales protegen y coordinan el acceso a datos compartidos. Estos mecanismos organizan la concurrencia, pero un Linux convencional no garantiza por sí solo que cada activación ocurra exactamente en el instante previsto.

2. Sistema de control de un automóvil

2.1. Diseño y configuración temporal

El programa simula cuatro funciones en cuatro hilos POSIX independientes: control de estabilidad, control de tracción, cálculo de combustible con actualización de velocidad y diagnóstico de telemetría. Un instante de referencia común basado en `CLOCK_MONOTONIC` permite definir los desfases iniciales de las tareas.

Tabla 1: Tareas periódicas, mecanismos de activación y parámetros temporales.

Tarea	Función	Mecanismo	Período	Desfase
Tau 1	Estabilidad ESC	Temporizador y señal RT	20 ms	5 ms
Tau 2	Tracción TCS	Temporizador y señal RT	40 ms	25 ms
Tau 3	Combustible y velocidad	Reloj, espera absoluta	80 ms	45 ms
Tau 4	Diagnóstico	Reloj, espera absoluta	160 ms	70 ms

Los períodos de Tau 1, Tau 2 y Tau 3 coinciden con los valores de la guía; el período de Tau 4 se definió en 160 ms para el diagnóstico. Los desfases son decisiones de diseño. Al iniciar el programa se reserva un breve margen para crear los hilos y luego todas las fechas iniciales se calculan desde el mismo epoch.

2.2. Cuerpos de las tareas y modelo simulado

- **Tau 1 – Estabilidad:** genera una aceleración lateral simulada que normalmente es $0,22g$, y sube a $0,93g$ durante la fase de maniobra de cada ciclo de cinco segundos. Si supera el umbral de $0,65g$, solicita frenado proporcional y activa una alerta.
- **Tau 2 – Tracción:** simula un patinamiento del 24 % durante una fase de cada ciclo de seis segundos y del 5 % en el resto. A partir de ese valor y del frenado solicitado calcula el par motor. Después publica una muestra del estado compartido en la cola de telemetría.
- **Tau 3 – Combustible y velocidad:** estima el nivel de combustible a partir del par, la velocidad y el frenado. Si la alerta de estabilidad está activa, reduce el valor calculado. También actualiza la velocidad con un modelo discreto simplificado.
- **Tau 4 – Diagnóstico:** consume las muestras de telemetría disponibles y presenta la más reciente junto con el número de muestras procesadas desde el diagnóstico anterior.

Las operaciones son una simulación de propósito académico, no un modelo físico validado ni una estrategia apta para controlar un vehículo real.

El arreglo compartido contiene cuatro señales: frenado solicitado, par motor, nivel de combustible y velocidad. Sus valores iniciales son 0 % de frenado, 72 % de par, 58 % de combustible y 54 km/h. El cálculo de frenado usa el exceso de aceleración lateral sobre el umbral:

$$B = \text{limitar}(120(a_{\text{lat}} - 0,65), 0, 45),$$

donde a_{lat} se expresa en unidades de g , y B es el porcentaje de frenado solicitado.

El TCS limita el par calculado al intervalo de 28 % a 96 %:

$$T = \text{limitar}(96 - 2 \max(S - 10, 0) - 0,22B, 28, 96),$$

donde S es el patinamiento porcentual. La estimación de combustible es:

$$F = \text{limitar}(24 + 0,62T + 0,08V - 0,18B, 15, 90),$$

donde V representa la velocidad en km/h. Con la alerta activa, F se multiplica por 0,88. Finalmente, la velocidad se actualiza cada 80 ms mediante el modelo discreto:

$$V_{k+1} = \text{limitar}(V_k + (0,022T - 0,038B - 0,22)0,08, 0, 130).$$

Los coeficientes se eligieron para producir una respuesta acotada y observable en la simulación; no representan unidades ni parámetros de un vehículo real.

2.3. Recursos compartidos y sincronización

El estado global del vehículo contiene las cuatro señales, las variables de escenario, las métricas temporales y la cola de muestras. Un mutex protege las lecturas y actualizaciones de las señales y las variables de escenario.

La cola circular de telemetría dispone de un mutex propio y dos variables condicionales. El productor espera en `space_ready` si la cola está llena y el consumidor espera en `item_ready` cuando no hay datos. Al insertar o retirar una muestra, se notifica a la otra parte. Las comprobaciones se realizan en bucles asociados al mutex para tolerar despertares espurios. Al detener el programa se cambia el estado de ejecución bajo el mutex del vehículo y se despiertan los posibles hilos bloqueados en la cola.

2.4. Temporizadores, relojes y señales

El programa bloquea las señales de los dos temporizadores antes de crear los hilos. Tau 1 y Tau 2 configuran temporizadores con `timer_create()` y `timer_settime()`, usando `TIMER_ABSTIME` para basar la primera expiración en el epoch común y un intervalo para las siguientes. Cada hilo espera su señal con `sigwait()` y ejecuta su cuerpo tras cada notificación. Al terminar, el hilo principal cambia el estado de ejecución y envía a esos hilos la señal correspondiente para liberarlos de la espera.

Tau 3 y Tau 4 calculan su primera fecha de activación a partir del mismo epoch. En cada ciclo esperan con `clock_nanosleep()` y `TIMER_ABSTIME`, y avanzan la fecha objetivo exactamente en un período. Esto evita basar la siguiente fecha en el final del procesamiento anterior.

2.5. Medición y límites de la simulación

Cada hilo registra el número de activaciones, el período medio observado y la desviación máxima respecto al período nominal (jitter). Estas mediciones permiten comparar el comportamiento observado con los períodos configurados, pero no constituyen una prueba formal de determinismo ni una garantía de tiempo real duro. Los resultados dependen de la carga y de la planificación del sistema anfitrión.